HOME BAKERS PLATFORM

Master AI Coding-Agent Completion Specification

Customer/User App + Admin App + Backend + PostgreSQL

Purpose: Give this document to the AI coding agent and instruct it to finish the existing project end-to-end. The agent must work with the existing code, preserve working functionality, and implement/fix the remaining functionality instead of blindly rebuilding the applications.

Source basis: The three uploaded merged codebases were reviewed: Cakedeliveryapp (customer), cakedeliveryadmin (admin), and Backendofbakersapp (backend/database). The source material shows an existing but partially integrated system with React Native apps, Express/Prisma/PostgreSQL backend, authentication, products, cart, orders, addresses, profile, analytics, notifications, reviews, UPI stub, onboarding and image upload routes.

Important: This specification is a completion contract. The AI agent must inspect the actual repository before changing code and must verify every item with the running applications/backend. If an item already works correctly, do not rewrite it unnecessarily.

1. CURRENT SYSTEM UNDERSTANDING

- There are three separate codebases: customer mobile app, admin mobile app, and backend server.
- The customer app already has navigation, authentication screens, home/category/product flows, custom order UI, cart/checkout UI, address/profile/order/tracking/notification screens, but several flows historically relied on mock data, local state, AsyncStorage, or fake success behavior.
- The customer analysis reports an estimated ~85% functional customer journey, while explicitly stating that production backend integration was not completed in that earlier customer-only scope.
- The backend exposes customer and admin routes for auth, products, cart, orders, addresses, profile, analytics, notifications, reviews, UPI, store profile, onboarding and product-image upload.
- The database contains users, addresses, cart, products, orders, order items, payments, reviews, notifications and delivery tracking structures, with PostgreSQL/Prisma used by the backend.
- The admin app already consumes live analytics/orders/catalog in places, but contains hardcoded API base URLs and some derived/simulated dashboard presentation logic.

2. TECHNICAL BASELINE

Layer	Observed baseline	Completion direction
Customer	React Native; screens/components; AsyncStorage/service layer exists	Connect every business flow to backend and remove mocks where backer
Admin	React Native; dashboard/catalog/security/profile-style pages	Connect every management action to secured backend APIs
Backend	Node.js/Express 5, Prisma 7, PostgreSQL, bcrypt, JWT, helmet, cors	Harden, authenticate, validate, complete business logic
Database	PostgreSQL schema + Prisma models/migrations	Reconcile Prisma model names/relations with actual database and applica

3. CUSTOMER APP — REQUIRED COMPLETION

Authentication: Make signup/login/reset-password production-ready; persist auth state; use one configurable API base URL; remove hardcoded LAN IPs; show proper loading/error states; ensure logout clears local session.

Home / Catalog: Replace mock product/catalog data with backend data where equivalent endpoints exist; implement search/category/filter/sort/refresh consistently; ensure product IDs and fields match backend.

Product Details: Load real product details, quantity, availability, favorites if supported by current design, reviews where backend supports them, and real add-to-cart.

Cart: Use /api/cart endpoints; fetch persisted cart after login; add/update/remove correctly; prevent invalid quantities; calculate totals from trusted product/backend data.

Address: Use address CRUD endpoints; load saved addresses; create/update/delete/select default address; validate fields; prevent checkout without a usable address.

Checkout / Orders: Replace fake setTimeout checkout with real order creation. Send cart, selected address, payment method and required order information to backend. Handle success/failure idempotently.

Order History: Replace hardcoded order array with customer orders endpoint; show active/past statuses based on actual backend values.

Tracking: Use real order tracking endpoint; render current status and tracking information; refresh safely without excessive polling.

Notifications: Use notification APIs; unread/read state must persist; mark individual notifications read.

Profile: Load and update profile through backend; integrate password change; handle authentication failures.

Custom Orders: Complete custom-order request flow according to existing screens/backend capability. Do not invent unsupported backend contracts; if backend support is absent, add a coherent endpoint/model only after inspecting schema and existing architecture.

UI / Navigation: Fix inconsistent route names, broken imports, missing props and invalid navigation paths. Preserve current visual design unless a functional correction requires changes.

4. CUSTOMER BUGS/KNOWN ISSUES TO VERIFY

- Header contains a misspelled Notificaton.js import and unclear Blog navigation usage.
- CategoryListing and MenuCard have mismatched props; menu data has missing fields such as price/bakingTime in the previously identified implementation.
- Navigation names are inconsistent, including Categorys/Category and Adressscreen/AddressUI.
- Checkout previously navigated through an incorrect Blog → Adressscreen path.
- Signup/login/address historically used hardcoded IP addresses and lacked robust network failure handling.
- Checkout previously used a fake timeout rather than guaranteed real order creation.
- Several screens historically contained hardcoded/mock data.

5. ADMIN APP — REQUIRED COMPLETION

Admin authentication: Implement secure admin login/session handling. Never rely only on hiding screens; backend must enforce admin authorization.

Dashboard: Use backend analytics as source of truth. Remove simulated chart values where real data can be calculated. Ensure revenue/order/product/review/low-stock cards reflect actual database state.

Catalog management: List products from backend; create/edit/delete/toggle availability; upload product image; validate fields; refresh after mutations.

Product fields: Keep frontend field names aligned with backend Product model: product name, description, image, price, weight/unit, stock, prep time, sizes, eggless, flavor, category, public catalog, bestseller, featured, custom-message capability.

Orders: List all orders; open order details; update valid statuses; show customer/address/items/payment information; reflect changes immediately.

Delivery tracking: Allow authorized admin operation to update tracking information/status; ensure order/customer sees the same state.

Customers / profiles: Where the existing admin UI supports it, show real user/profile information from backend. Do not expose password hashes.

Reviews: Display real product reviews/ratings where admin UI supports them. Do not fabricate review data.

Notifications: Allow appropriate admin-triggered notifications only through backend and authorization rules.

Store onboarding/profile: Complete store profile/onboarding persistence and ensure customer-facing store information can be read through the public store endpoint.

Security settings: Existing security toggles/delete-account UI must not pretend to perform backend actions. Connect them where backend support exists or clearly implement the required backend contract.

6. ADMIN APP ISSUES TO VERIFY

- Dashboard currently uses http://10.0.3.1:3000 directly for analytics, orders and catalog; replace with environment/config-driven API base URL.
- Catalog currently uses direct fetch calls to the same hardcoded base URL for delete and availability toggle.
- Dashboard sales-overview presentation contains simulated fallback values; use real aggregated backend data instead.
- Status terminology in dashboard logic includes values such as pending/preparing/out_for_delivery/completed, while the SQL schema has a different constrained status vocabulary. The agent must reconcile the actual API/database contract rather than patching one screen only.
- Security page includes local toggles and an account-deletion confirmation that may not actually call a backend operation.

7. BACKEND/API COMPLETION

The backend already exposes the following route groups; the agent must inspect each controller implementation and verify request/response contracts against both apps.

Group	Routes observed
Auth	POST /api/auth/signupmain; POST /api/auth/adminsignup; POST /api/auth/loginmain; POST /api/auth/loginadmin
Products	GET /api/products; GET /api/products/search; GET /api/products/category; GET /api/products/:id; POST /api/products; PUT /api/produc
Cart	POST /api/cart/add; GET /api/cart; PUT /api/cart/:id; DELETE /api/cart/:id
Orders	POST /api/orders; GET /api/orders; GET /api/orders/customer; GET /api/orders/:id; PUT /api/orders/:id/status
Tracking	GET /api/orders/:orderId/tracking; PUT /api/orders/:orderId/tracking
Address	POST /api/address/save; GET /api/address; PUT /api/address/:id; DELETE /api/address/:id

Group	Routes observed
Profile	GET /api/user/profile; PUT /api/user/profile/:id; PUT /api/user/change-password
Analytics	GET /api/dashboard/analytics
Notifications	POST /api/notifications; GET /api/notifications; PATCH /api/notifications/:id/read
Reviews	POST /api/reviews; GET /api/reviews
Payment stub	POST /api/upi/save
Store	GET /api/store
Onboarding	POST /api/onboarding/save
Image upload	/api/add/itemdata mounted upload route

8. BACKEND SECURITY AND CORRECTNESS

- Add/verify authentication middleware for protected routes. The presence of a Jwtverify helper alone is not enough; protected endpoints must actually verify tokens.
- Separate customer authorization from admin authorization. A customer token must not be able to mutate catalog, all-orders data, analytics, or other admin-only resources.
- Never log passwords, tokens, or sensitive credentials.
- Passwords must be hashed with bcrypt and never returned in API responses.
- Validate IDs, quantities, prices, statuses, addresses and required fields server-side.
- Do not trust client-provided total amount. Recalculate order totals from current product prices and server-side rules.
- Use database transactions for order creation and related order-item/cart/payment state changes where appropriate.
- Prevent duplicate order creation caused by repeated taps/network retries.
- Validate allowed order and payment statuses against the database/application contract.
- Add consistent HTTP status codes and JSON response shapes; frontend should not have to guess whether success occurred.
- Configure CORS and environment variables appropriately for development and production.
- Add centralized error handling rather than exposing raw internal errors to clients.

9. DATABASE / PRISMA RECONCILIATION

- The uploaded schema includes addresses, cart, delivery_tracking, discounts, user/login data, notifications, order_items, orders, payments, products and reviews.
- The Prisma schema maps User to the database table Login&signupsystem and includes Address and Cart relations.
- Product, Order, OrderItem, Payment, Review and DeliveryTracking relations must be inspected end-to-end.
- The database schema uses order_status values Pending, Accepted, Preparing, Ready, Delivered, Cancelled and Rejected. The agent must ensure frontend/backend use the same canonical values or implement an explicit mapping.
- Payment status is constrained to Pending, Paid, Failed and Refunded in the supplied SQL schema.
- Do not casually rename database tables/columns or reset production-like data. Prefer additive migrations and backward-compatible changes.
- Run Prisma validation/generation/migrations only after reviewing the current database state.

10. MASTER AI AGENT PROMPT — COPY THIS ENTIRE SECTION

```
You are the lead engineer responsible for completing my existing Home Bakers platform.

PROJECTS:

1) Customer/User React Native app: Cakedeliveryapp

2) Admin React Native app: cakedeliveryadmin

3) Backend Node.js/Express + Prisma + PostgreSQL: Backendofbakersapp

OBJECTIVE:

Finish the ENTIRE existing system end-to-end. Do not rebuild everything from scratch. First inspect the repository and understand the existing architecture, screens, navigation, components, API contracts, Prisma schema, migrations, controllers, routes and current implementation. Preserve working UI and business logic unless a change is required to make the system correct.

NON-NEGOTIABLE RULES:

- Work from the existing codebase.

- Do not invent API contracts when an existing endpoint/model already provides the required capability.

- Do not replace working features with mock implementations.

- Remove mock/fake success behavior wherever a real backend capability exists.

- Do not use hardcoded LAN/IP API URLs. Create/use one environment/config-driven API base URL per app.

- Do not expose passwords, password hashes, JWT secrets or sensitive server information.

- Do not trust client totals/prices/statuses when the server can calculate/validate them.

- Protect admin endpoints with real backend authorization.

- Protect customer data so one customer cannot access another customer's cart/orders/address/profile.

- Do not delete or reset database data just to make tests pass.

- Make database changes through safe Prisma migrations when needed.

- Keep naming consistent across frontend, backend and database.

- After every major change, run the relevant lint/type/build/test checks and fix errors before moving on.

- Do not stop after finding the first error. Continue until the whole system is coherent.

PHASE 1 — AUDIT BEFORE CODING

1. Inspect all three projects.

2. Identify package versions, entry points, navigation structure, API layers, controllers, routes, Prisma models and migrations.

3. Build an internal matrix:

Feature | Customer UI | Admin UI | Backend route | Controller | DB model/table | Current state | Required fix

4. Search for:

TODO, FIXME, mock data, hardcoded URLs/IPs, setTimeout fake success, Alert-only mutations, console logs containing credentials, inconsistent route names, undefined navigation targets, broken imports, mismatched request fields, mismatched response fields.

5. Do not assume the old analysis report is perfectly current; verify against the actual code.

PHASE 2 — BACKEND FOUNDATION

1. Make the backend start reliably.

2. Verify DATABASE_URL/DB_* environment configuration.

3. Verify Prisma client generation and schema validity.

4. Add/verify centralized error handling.

5. Add/verify authentication middleware using the existing JWT approach.
```

6. Implement explicit authorization roles/permissions for admin vs customer.

7. Standardize API responses:

{ success: boolean, message?: string, data?: any, error?: string }

8. Validate request bodies, params and query strings.

9. Never return Password fields.

10. Never log plaintext passwords or JWTs.

PHASE 3 — AUTHENTICATION

Customer:

- signup
- login
- logout
- session persistence
- password reset flow if the UI requires it
- change password

Admin:

- admin signup only if intended by existing architecture
- admin login
- protected session
- logout
- backend role enforcement

Ensure the frontend stores only the minimum required session information and sends the access token consistently.

PHASE 4 — PRODUCT/CATALOG

Backend:

- list products
- get product
- search
- category filter
- create
- update
- delete
- availability toggle
- image upload
- validate price/stock/prep time/weight fields
- support existing product flags

Customer:

- real catalog
- real search/filter/category
- real product detail
- real availability
- real add to cart

Admin:

- real catalog list
- real create/edit/delete

- real image upload
- real availability toggle
- refresh after mutations
- correct product IDs and field mappings

PHASE 5 — CART

Use backend cart endpoints.

Requirements:

- add product
- fetch current user's cart
- increment/decrement
- update quantity
- remove item
- persisted cart after app restart/login
- validate quantity > 0
- handle unavailable/out-of-stock products
- prevent unauthorized access to another user's cart

PHASE 6 — ADDRESS + PROFILE

Customer:

- fetch addresses
- create
- edit
- delete
- select/default address
- profile fetch/update
- change password

Admin:

- only show data that the current admin is authorized to see.

Do not leave UI buttons that claim success without a backend operation.

PHASE 7 — ORDER CREATION

Implement a real transactional order flow:

1. Authenticate customer.
2. Load cart from DB.
3. Load products from DB.
4. Validate availability/stock.
5. Recalculate subtotal server-side.
6. Apply legitimate delivery/tax/discount rules only if supported.
7. Validate selected address.
8. Create order + order items.
9. Create/update payment record if appropriate.
10. Clear cart only after successful order creation.
11. Return order ID/order number and status.
12. Make retry behavior safe against duplicate orders.

Customer:

- checkout
- success screen
- order history
- order detail
- status
- tracking

Admin:

- all orders
- order details
- status update
- delivery/tracking update

IMPORTANT:

The supplied database has canonical order statuses:

Pending, Accepted, Preparing, Ready, Delivered, Cancelled, Rejected.

Do not silently use different values such as "pending", "completed" or "out_for_delivery" unless the backend explicitly maps them. Choose one canonical contract and update every layer consistently.

PHASE 8 — DELIVERY TRACKING

- Store/retrieve tracking data through backend.
- Ensure tracking belongs to the requested order.
- Ensure customer can only see their own order tracking.
- Ensure admin can update tracking.
- Keep order status and tracking status logically consistent.
- Do not fabricate courier data.

PHASE 9 — PAYMENTS

The existing /api/upi/save endpoint is only a stub. Inspect the current payment UI and decide the minimum safe implementation needed for this project.

If real payment gateway integration is not present, do NOT pretend a payment is successful.

Implement explicit states such as Pending/Paid/Failed/Refunded according to the database contract.

If a real gateway is required but credentials/configuration are absent, implement the integration boundary and clear configuration requirements without hardcoding secrets.

PHASE 10 — NOTIFICATIONS + REVIEWS

Notifications:

- create when appropriate
- list for correct user
- mark read
- unread state persists

Reviews:

- customer can create a review only where permitted
- validate rating range
- list reviews by product
- connect product detail UI to real reviews if the current UI supports it
- prevent unauthorized user identity spoofing

PHASE 11 — ADMIN DASHBOARD

Replace fake/simulated data with real backend values.

Dashboard should correctly show:

- total revenue
- total orders
- active orders
- pending orders
- delivered orders
- total products
- low stock
- recent reviews
- recent orders
- best sellers
- sales overview

Do not derive incorrect "today" values using UTC if the intended business day is local time without considering timezone.

Do not show made-up fallback chart values when there is insufficient data; show zero/empty state with a correct message.

PHASE 12 — ADMIN CATALOG

Complete:

- list
- search
- category filtering
- sorting
- create
- edit
- delete
- availability
- image upload
- validation
- optimistic/local state updates only after successful backend mutation
- proper loading/error/empty states

PHASE 13 — NAVIGATION / UI CLEANUP

Fix all broken navigation and imports.

Known items to verify include:

- Notificaton.js misspelling/broken import
- Category vs Categorys naming
- AddressUI vs Adressscreen naming
- invalid Blog navigation used as a profile/address route
- MenuCard prop mismatch
- missing product fields
- any undefined navigation targets
- any screen that exists but is unreachable

Do not redesign the application unnecessarily. Keep the existing visual identity.

PHASE 14 — NETWORK CONFIGURATION

Create a clean API configuration strategy:

- development Android emulator

- physical Android device

- production/server deployment

No hardcoded:

10.x.x.x

192.168.x.x

localhost

or other machine-specific URLs inside business screens/components.

Use environment/configuration and document exactly where the developer changes the API URL.

PHASE 15 — TESTING

Test the complete journey manually and, where practical, automatically.

CUSTOMER E2E:

signup → login → home → search/category → product detail → add to cart → cart update → address → checkout → order success → order history → tracking → notification → profile

ADMIN E2E:

admin login → dashboard → catalog → add product → upload image → edit product → availability → orders → order detail → update status → tracking → dashboard refresh

CROSS-APP TEST:

customer creates order

→ database receives order

→ admin sees order

→ admin changes status

→ customer sees changed status

→ admin updates tracking

→ customer sees tracking

→ notifications are created/read correctly.

NEGATIVE TESTS:

- invalid login

- duplicate signup

- expired/invalid token

- customer calling admin endpoint

- customer requesting another customer's order

- invalid product ID

- quantity 0/negative

- out-of-stock purchase

- invalid order status

- malformed address

- failed network request

- duplicate checkout tap

- missing image/file

- invalid review rating

PHASE 16 — BUILD/LINT/ERROR CLEANUP

Run appropriate commands for each project, such as:

- npm install / npm ci as appropriate
- Prisma validation/generation/migration checks
- backend startup
- frontend Metro/build
- Android debug build where applicable
- lint/tests where configured

Fix:

- compile errors
- import errors
- navigation errors
- Prisma errors
- API 4xx/5xx caused by your implementation
- undefined fields
- incorrect response parsing
- warnings that indicate real bugs

PHASE 17 — FINAL VERIFICATION

Before declaring completion:

1. Search again for hardcoded API URLs.
2. Search again for fake setTimeout checkout/success.
3. Search again for mock product/order arrays used in production paths.
4. Search for unprotected admin routes.
5. Search for Password fields being returned/logged.
6. Search for inconsistent order status strings.
7. Search for broken navigation names.
8. Search for TODO/FIXME in business-critical paths.
9. Verify customer/admin/backend contracts.
10. Verify database relations and migrations.
11. Verify fresh install/startup instructions.
12. Produce a final completion report.

FINAL REPORT FORMAT:

A. Completed features

B. Files changed

C. Database/migration changes

D. API endpoints verified

E. Security fixes

F. Tests executed and results

G. Known limitations requiring external credentials/services

H. Exact commands to run customer app, admin app and backend

I. Any remaining task that truly cannot be completed because required external information is unavailable

DEFINITION OF DONE:

The project is done only when the three codebases behave as one coherent application:

Customer actions persist to the database;

Admin actions affect customer-visible state;

API contracts match frontend expectations;

authentication/authorization is enforced;

orders and statuses are consistent;

no critical flow relies on fake/mock success;

and the applications build/run without critical errors.

Start by performing the audit and creating the feature matrix. Then implement the fixes phase-by-phase. Do not stop at analysis: actually modify the code, test it, and continue until the Definition of Done is satisfied.

11. ACCEPTANCE CHECKLIST

- [ ] Customer signup/login works against backend and survives app restart.
- [ ] Admin login works and admin-only backend resources reject customer credentials.
- [ ] Catalog is database-driven in customer and admin apps.
- [ ] Product create/edit/delete/availability/image upload work end-to-end.
- [ ] Cart is persisted in PostgreSQL and scoped to the authenticated customer.
- [ ] Address CRUD and default address work.
- [ ] Checkout creates a real database order and clears the cart only on success.
- [ ] Server recalculates order totals.
- [ ] Customer sees their own orders only.
- [ ] Admin sees and updates orders.
- [ ] Order status values are consistent across DB/backend/admin/customer.
- [ ] Tracking is shared correctly between admin updates and customer view.
- [ ] Payment state is never falsely reported as paid.
- [ ] Notifications persist and read state works.
- [ ] Reviews use authenticated identity and validated ratings.
- [ ] Dashboard numbers come from real data and do not contain fabricated fallback business values.
- [ ] No hardcoded developer LAN IP remains in business/API code.
- [ ] No password or secret is logged or returned.
- [ ] All critical navigation targets exist and work.
- [ ] Backend and both apps can be started using documented commands.
- [ ] Final build/lint/test checks pass or any unavoidable external limitation is explicitly documented.

12. IMPORTANT SOURCE NOTES FOR THE AGENT

- The customer source previously documented real-auth persistence, a customerApi service layer, cart persistence and an integrated customer journey, but also explicitly said the backend was excluded from that earlier customer-only implementation. Treat those as clues to inspect, not as proof that production integration is complete.
- The backend route file already imports controllers for auth, addresses, profile, analytics, onboarding, notifications, reviews, cart, orders and products. Verify every controller rather than creating duplicate routes.
- The supplied Prisma configuration uses DATABASE_URL and the backend package includes Express, Prisma, PostgreSQL, bcrypt, JWT, multer, QRCode and WebSocket-related dependencies.
- The supplied SQL schema has strict constraints for order status, payment status, nonnegative prices/stock, and several foreign keys. Preserve these business constraints.

End of specification.

Use this PDF as the master task instruction. The AI agent should still inspect the actual repositories before making changes.